20 HOURS. ONE CHAMPION.

CODE BOT

Rules & participant guide

Build an autonomous bot for the generals.bot competition ruleset, then put it to the test against the other participants.

Welcome to Code Bot

Version 2.0 • Issued 30 September 2026. Join us on 2 October 2026 for an individual bot-building hackathon. You have 20 hours to develop one Python bot. We then run the tournament offline and award US$500, US$300 and US$200 to the top three finishers. Exact start time, venue and submission channel will be announced by the organizer.

Event detail	Event details
Host / event date	Animish Sharma / 2 October 2026
Venue / help channel	[VENUE OR ONLINE] / [HELP CHANNEL]
Build window	2 Oct 2026 [START TIME, TIMEZONE] to [END DATE/TIME, TIMEZONE] — exactly 20 hours
Participation / field	Individual participation; one Python file per person
Submission / tournament	[SUBMISSION LINK] / [TOURNAMENT START AFTER H20]
Awards / dispute contact	1st US$500 / 2nd US$300 / 3rd US$200; [DISPUTE CHANNEL]

At a glance

Each participant submits one .py file implementing act(observation). Development closes at H20. A two-game round robin seeds the top four into the semifinals. The final determines the champion; a third-place playoff determines bronze. Official standings depend only on match results, subject to eligibility and integrity rulings.

How to use this document

These are the participant rules for Code Bot. Pages 2–7 explain the event, fair play, runtime and scoring. Page 8 defines the Python API. Our published evaluator and starter accompany this document. Fields awaiting an announcement are shown in brackets.

1. Event format & clock

Code Bot is an independent hackathon using the published generals.bot competition ruleset. Each participant develops one bot; an official game is a 1v1 contest between two bots. A series contains several games.

1.1 Twenty-hour development window

T0 is the published kickoff. T0 + 20:00 is the submission deadline. The tournament, validation and awards take place afterward. Organizer server time is authoritative; publish both exact timestamps and their timezone. No strategy, parameter, model-weight or behavior changes are permitted after the deadline.

Time	Suggested milestone
Before T0	Registration, tool installation and starter-kit access; no event-specific strategy development.
H00–H02	Run the starter and complete a baseline match.
H02–H08	Implement an initial strategy.
H08–H16	Evaluate improvements across varied maps.
H16–H18	Freeze a candidate and test it in the event image.
H18–H20	Submit, verify the receipt and retain a known working Python file.
After H20	Validate frozen entries, publish seeds, run the tournament and awards.

1.2 Eligibility & participants

Participation is individual. Each registered person may submit only one official bot and must develop their own event strategy. Registration identity locks at T0. Participant handles and bot names must be respectful. Registration closes at [REGISTRATION DEADLINE]. At least two eligible entries are required for a competitive tournament. We will publish the tournament timetable after registration.

1.3 Official communication

The announcement channel is [ANNOUNCEMENT CHANNEL]. Organizers publish clarifications there for everyone. Material rules, resource limits and baseline assets must be fixed before kickoff. An organizer-wide outage may pause the clock or cause an equal extension for all participants; personal equipment failures do not automatically extend a deadline.

2. Game rules & version control

We use the bundled competition engine at commit 13db8f69a422380ea184d2f4ca262a38866c5fc6 with competition mode enabled. That frozen implementation resolves game mechanics. This document governs the event format, submission and conduct. Any conflict must be clarified publicly before official play; do not silently substitute a newer upstream version.

2.1 Mechanics summary

Capture the opposing general to win. Competition boards are 18–21 cells per side, with fog of war. Mountains block movement. Castles are built, not initially neutral. Actions move to an orthogonal neighbor, build or pass. Moves send half an army or all but one. Combat subtracts armies; equal strength leaves ownership with the defender.

Generals and castles grow every other turn; owned land grows every 50 turns. A castle costs 35 plus the sum of max(0, 14 − 2d) over your structures, where d is Manhattan distance. Build costs are paid on the target tile.

Deathtouch starts at turn 800: an attack that executes onto the enemy general wins regardless of defense. Intercepted attacks may fail to execute. Unresolved games draw at turn 1200; simultaneous general captures also draw. Territory and army totals do not break these draws.

Full movement, visibility and build semantics — https://www.generals.bot/rules

2.2 Legal information

Bots may use the supplied observations, public rules, their own internal state and approved packaged assets. Memory of earlier observations is allowed. Reading opponent files, replay feeds, hidden board state, secret seeds, server internals or other unapproved information during official games is prohibited.

2.3 Testing & map fairness

Practice results do not count toward standings. Organizers keep official maps and seeds private until the relevant games end. Each game uses a fresh generated map; starting slots are exchanged within each two-game pair. Seeds, engine version and results are retained for audit. Slot exchange reduces systematic starting-position bias but does not make different maps identical.

2.4 Do not rely on the live public leaderboard

Only games scheduled and recorded by event officials count. Public-platform ratings, previous competition results and participant-run scrimmages have no effect on this event's standings. The event submission channel and runner will be announced by the organizers.

3. Originality, AI tools & fair play

3.1 Work during the event

Develop the bot's event-specific decision logic, tuned parameters and trained policy during the 20-hour window. Before T0, you may install tools, study public material and verify an unmodified organizer starter. Generic utilities and organizer-provided starter code are allowed with attribution, provided the final file uses only Python's standard library.

Do not submit a pre-existing complete competitive bot or pretrained game policy as your entry. An exception is valid only if organizers explicitly list that artifact as a shared starting asset before kickoff. No external model files, downloaded assets or third-party Python packages may accompany or be fetched by the submission. Any approved small constants must be contained in the single source file and attributed.

3.2 AI-assisted development

AI coding assistants, chat tools and code generation are allowed during development. Disclose the tools used and material contributions in the source-file header. Participants remain responsible for correctness, licensing and understanding their submitted code. Each participant must be able to explain their strategy and reproduce its launch. An online AI service cannot make decisions during an official game.

3.3 Collaboration & attribution

Each participant works independently. Public conceptual discussion and organizer help are encouraged. Do not exchange event strategy code, weights or private tactical assistance between competing participants during the build window. Cite external code, libraries, models and substantial borrowed ideas. Mentors may explain concepts and troubleshoot setup; they may not build a participant's strategy for it.

4. Prohibited behavior

No human control after an official game starts; no changes to the frozen artifact; no collusion, match fixing or deliberately favoring another participant. Do not tamper with the runner, exploit a sandbox escape, bypass visibility restrictions, attack infrastructure or inspect another participant's files. Report suspected platform vulnerabilities privately to officials and stop using them.

Harassment and disruption are not acceptable. Organizers may issue a warning, void or forfeit affected games, or disqualify a participant according to the evidence and severity. Affected participants may provide an explanation before a final disqualification ruling. Officials record the reason and any standings adjustment.

Code ownership & disclosure

Participants retain ownership of their original work. Submitting grants organizers permission to inspect and run the entry for this event and its result review. Public release of source code requires the participant's consent. Match results and replays may be shown as part of the tournament.

5. Runtime & single-file submission

Submit one UTF-8 Python file named participant_id.py. It must expose def act(observation) and return one action. We run it with our provided adapter; a main loop, stdin parser, run.sh, build script, ZIP or README attachment is not required.

Constraint	Code Bot event requirement
Language	CPython 3.12; standard library only
Source size	One .py file, at most 1 MiB (1,048,576 bytes)
Decision time	150 ms per response; first response gets 10 s including import
Compute	One selected logical CPU; bots take turns on the same CPU
Memory / storage	2 GiB per bot; temporary scratch space 16 MiB
Connectivity	Network disabled; no GPU, external services or package installation
Game failure	Timeout, exception, crash or malformed return: immediate forfeit
Illegal game action	Valid five-integer action that is illegal on the board: pass

5.1 Bot lifecycle

We start a new isolated process for each game. Your module is imported on its first call; module globals persist between turns of that game. The inactive bot is paused. Both bots receive observations of the same board state before actions resolve. Request order alternates. The source is read-only; no other submission or engine files are mounted in the bot container.

The response clock includes delivering the observation, Python execution and receiving the action. Container startup and engine compilation are excluded. The first response includes your import time. A runtime/protocol failure forfeits that game immediately; the website's 50-fault allowance does not apply to Code Bot. A double forfeit awards zero points to both.

5.2 File header & submission receipt

In a top-level comment or docstring, include your registered ID, bot name, a short strategy explanation, reused sources and material AI assistance. Include all strategy code in this file. Do not include credentials. Submit it to [SUBMISSION LINK]. The latest complete file received by H20 is your entry; keep the receipt. No strategy or packaging changes are accepted after H20.

We validate the file and record its SHA-256 checksum. Syntax or API errors and failure to launch make an entry ineligible. Successful intake does not excuse later match-time faults. Static validation alone is insufficient: run sandboxed practice games before submitting.

6. Tournament, scoring & tiebreaks

6.1 League stage

With four or more eligible entrants, run a complete round robin: each unordered pair plays two games, one in each starting slot, using fresh maps. Score each game as win = 1, draw = 0.5, loss = 0. Forfeits award 1 to the opponent and 0 to the forfeiting participant; a double forfeit awards 0 to both. Sum game points to form league standings. No pitch, code-style or audience score contributes.

6.2 League ties

Order tied participants by: (1) points earned in games among that tied set, (2) total league wins, then (3) a reproducible seeded draw. Apply these criteria once in order; do not recalculate mini-leagues for smaller tied subsets. Publish the draw seed, ordering algorithm and stable participant IDs before official results are known. Retain the draw output in the event record.

6.3 Knockouts

Stage	Pairing and games
Semifinals	League seed 1 vs 4; seed 2 vs 3. Four games per series.
Final	Semifinal winners play six games for first and second place.
Third-place playoff	Semifinal losers play four games for third place.
Any tied series	Play one additional two-game pair. If still tied, the higher league seed wins the series.

Every two-game pair exchanges starting slots and uses fresh maps. Play all scheduled games; the higher series point total wins. Draws count as 0.5 each. The league seed is a deliberate, published final tiebreak, including in the championship. Semifinal losers play a four-game third-place playoff under the same series rules and tiebreak. Its winner takes third place.

6.4 Small fields & withdrawals

With two or three eligible entrants, keep the two-game round robin and send the top two directly to the six-game final. With three entrants, the remaining entrant places third. With two entrants, no third-place prize is awarded. With fewer than two, run an exhibition and do not award a competitive champion. The eligible field locks before the league begins. Withdrawal before that lock removes the entry. After the lock, a voluntary withdrawal leaves the frozen bot in the schedule. Integrity disqualifications require a referee ruling and a recorded standings correction; affected advancement is paused until resolved.

6.5 Capacity

For N league participants, the schedule contains N × (N − 1) games. At 16 participants that is 240 league games plus 18 scheduled playoff games, with up to eight additional playoff tiebreak games. Tournament start and expected finish are published after a runner dry run; they are separate from the 20-hour build clock.

7. Failures, disputes & results

7.1 Distinguish bot faults from infrastructure faults

A participant's timeout, protocol fault, crash or excess memory use forfeits the game under section 5. A bad decision follows the game rules and is not grounds for a replay. Officials may replay a game only when logs establish an organizer infrastructure or referee failure that invalidated it. Record the cause. Use the same frozen bots and configuration; replay the original seed if it remains private, otherwise use a fresh seed with the same starting slots.

If multiple games are affected, pause that portion of the schedule and apply the same remedy to all affected participants. Do not rerun unaffected games selectively. If an infrastructure incident prevents completion, pause and publish a resumption plan; do not invent a winner from territory or subjective strategy quality.

7.2 Appeals

A participant must submit a result dispute to [DISPUTE CHANNEL] within 10 minutes of that result's publication. Include participant IDs, game or series ID, the claimed rule violation and any supporting logs or screenshots. General disagreement with a loss is insufficient. Officials retain server logs and replay evidence and may request the submitted source.

Where a timely appeal could affect advancement, hold the affected next round until review is complete. A referee with no conflict of interest involving either participant reviews the evidence, allows a concise response from the affected participant and publishes a reasoned ruling. The head referee's event ruling is final. New integrity evidence may be considered after the ordinary appeal window.

7.3 Results & awards

Publish league points, tiebreak applications, bracket results and any penalties. The final-series winner is the champion; the finalist is runner-up. First place receives US$500, second place US$300 and third place US$200. A participant must remain eligible to receive a prize. The host will announce payment timing and method before kickoff. Optional special recognitions, such as best strategy explanation, must be labeled separately and cannot change competitive standings.

7.4 Participant checklist

☐ I am registered as an individual; I know the exact deadline and timezone.

☐ My bot runs offline in the event image and fits all limits.

☐ I tested complete matches and checked failure logs.

☐ My single .py file exposes act(observation) and includes the required header.

☐ My file header discloses reused material and AI assistance.

☐ I received a timestamped submission receipt before H20.

8. Your Python bot API

Your file is imported as a module. Define act(observation) at the top level. The input is a fresh ordinary Python dictionary. Return a list or tuple of exactly five signed 32-bit integers. Booleans, strings, floats, wrong lengths and invalid action codes are malformed returns and forfeit the game.

8.1 Observation dictionary

Keys	Meaning
turn, height, width	Current engine turn and board dimensions
player_id	Starting slot: 0 or 1; ownership codes remain relative to you
my_land, my_army	Your total territory and army
opp_land, opp_army	Opponent totals supplied by the engine
type	height × width grid: 0 fog, 1 plain, 2 mountain, 3 castle, 4 general, 5 structure in fog
owner	Matching grid: 0 neutral/unknown, 1 yours, 2 opponent
army	Matching grid: observed army counts; hidden cells masked by the engine

8.2 Return an action

Return [kind, row, col, direction, split]. Coordinates start at zero. Kind: 0 move, 1 pass, 2 build. Direction: 0 up, 1 down, 2 left, 3 right. Split: 0 sends all but one, 1 sends half. Direction and split must use valid codes even when ignored for pass or build. Out-of-board coordinates or otherwise illegal moves/builds waste the turn.

Minimal valid file:

```
def act(observation):
    return [1, 0, 0, 0, 0]
```

This bot always passes. Start from examples/starter.py for a simple expansion strategy. Global variables may store scouting memory within a game; all globals reset when the next game starts.

8.3 Resources

Underlying game rules — https://www.generals.bot/rules

Published platform documentation — https://www.generals.bot/docs

Pinned engine source — https://github.com/strakam/generals-bots/tree/13db8f69a422380ea184d2f4ca262a38866c5fc6

Our evaluator README contains installation and match commands. Code Bot uses its own single-file API, standard-library restriction and immediate-forfeit policy. Those event requirements replace the public platform's ZIP submission and fault-budget rules. The frozen engine resolves game-mechanics edge cases.